PDE-Based Image Reconstruction

James Rosado

Abstract

Computer Vision is an interdisciplinary field that deals with how computers extract information from digital images or videos. In particular one of the important tasks is how to process digital images and extract high dimensional data from the digital images [9, 14, 11]. For this project we will explore the active contour model, also called snakes, to delineate an image object from noise [7], we will apply our numerical algorithm to outline the contours of a spine dendrite.

Keywords: numerical partial differential equations, numerical linear algebra, level set methods, computer vision, active contours, image segmentation, computational neuroscience

1. Introduction

Image segmentation is the process of partitioning raw digital data (an image) into image objects, e.g. pixels, line segments, closed contours/surfaces. The purpose is to generate meaningful data e.g. a 2D closed contour or 3D triangulation that then can be utilized as a computational domain to analyze/solve PDE model equations on [15]. In this project we will construct contours that delineate the important landmarks of a noisy image, the process itself will require the use of non-linear equations PDEs. In particular one methodology utilizes active contours (or snakes) [13], that is an evolving curve which is constrained by a given raw image $u_0(x)$, where $x \in \Omega \subset \mathbb{R}^2$ in 2D as an example. The snake models that are used [13] are based on

$$F_1(C) = \alpha \int_0^1 |C'(s)|^2 \, ds + \beta \int_0^1 |C''(s)| \, ds - \lambda \int_0^1 |\nabla u_0(C(s))|^2 \, ds \tag{1}$$

where the contour C that minimizes the above functional is the contour that realizes the image and this is analogous to a non-linear PDE. The first two terms control the smoothness of the active contour and the last term moves the contour toward our image object. In order to determine the movement of the contour we use the maximum of $|\nabla u_0|$, that is the maximum of the image gradient and acts the *edge detector*. An edge detector is a positive decreasing function $g(\vec{z})$ depending on the gradient u_0 where

$$\lim_{|\vec{z}| \to \infty} g(\vec{z}) = 0.$$

For this investigation we use the following edge detector function

$$g(\nabla u_0(\vec{x})) = \frac{1}{1+\gamma|J\star\nabla u_0|^p} = \frac{1}{1+\gamma\left((J_\sigma\star u_0)_x^2+(J_\sigma\star u_0)_y^2\right)^{p/2}}, \tag{2}$$

where J_σ is the Gaussian Kernel with variance σ and γ, p are parameters we use to control the smoothing of the original image u_0. A more compact version of (1), [13], is given by

$$F_2(C) = \int_0^1 g(\nabla(u_0(s)))\ ds$$

which is analogous to

$$\begin{aligned}
\phi_t &= |\nabla\phi|\nabla\cdot\left[g(\nabla u_0)\left(\frac{\nabla\phi}{|\nabla\phi|}\right)\right] \\
&= |\nabla\phi|\left(g(\nabla u_0)\nabla\cdot\frac{\nabla\phi}{|\nabla\phi|}+\nabla g(\nabla u_0)\cdot\frac{\nabla\phi}{|\nabla\phi|}\right) \\
&= |\nabla\phi|\left(g(\nabla u_0)\kappa+\nabla g(\nabla u_0)\cdot\frac{\nabla\phi}{|\nabla\phi|}\right) \\
&= |\nabla\phi|g(\nabla u_0)\kappa+\nabla g(\nabla u_0)\cdot\nabla\phi
\end{aligned} \tag{3}$$

where ϕ is the level set function, κ is the curvature, of our image u_0, this formulation is given by [16], and our active contour is the level set of ϕ, $\phi = 0$. This equation will be the basis for our numerical scheme and demonstrates that the motion of the contour is given by the normal velocity times the curvature, in addition to a convection term. In the next section we will explain the Gaussian smoothing J_σ

2. Gaussian Smoothing

Our edge detector $g(\cdot)$ has input $J_\sigma\star\nabla u_0$, if we do not smooth the initial image then our image gradient we will be or order $\mathcal{O}(1/h)$ and this will cause our gradient to increase without bound as $h\to 0$. To remedy this there are two options (both equivalent): either convolve the initial image with the Gaussian Kernel function [13]

$$J_\sigma = J(x,y,\sigma) = \frac{1}{4\pi\sigma}e^{-(x^2+y^2)/(4\sigma)}$$

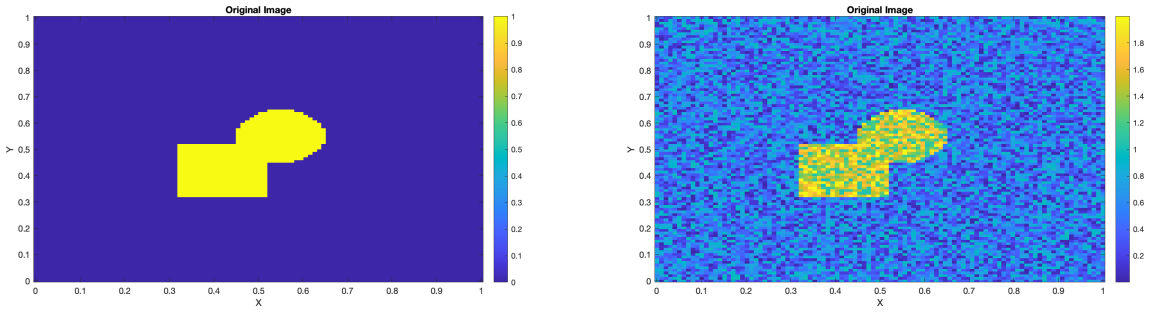
then

$$u_{init}(x,y) = \iint_\Omega J(x-x',y-y',\sigma)u_0(x',y')\ dx\ dy = (J_\sigma\star u_0)(x,y)$$

Or the other option is to solve the initial value problem for the heat equation [13] given by

$$\begin{cases} u_t = \nabla^2 u \\ u(x, y, 0) = u_0(x, y) \end{cases}$$

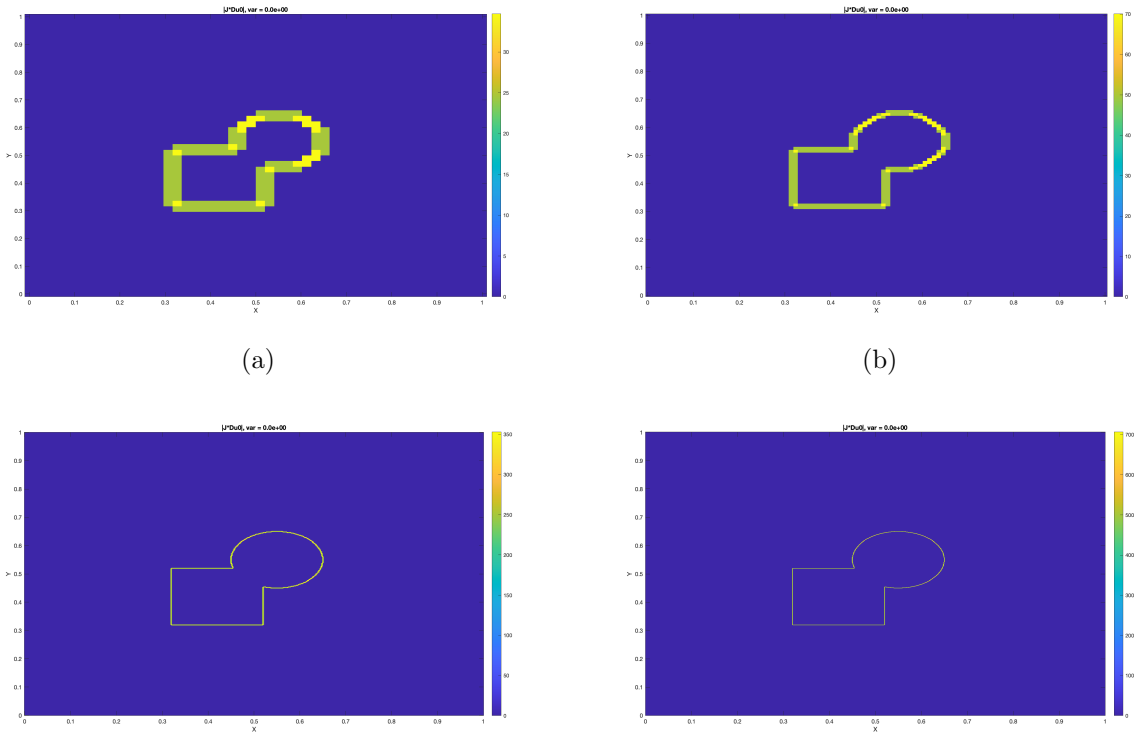
with Neumann boundary conditions and up to $t = \sigma > 0$. Below is our original test image and with noise:



(a) (b)

Figure 1: (a) original image, (b)noisy image

Below are a series of images that show $|\nabla u_0|$ for increasing resolution, without Gaussian smoother:



(a) (b)

(c) (d)

Figure 2: Resolutions (a) 50x50, (b) 100x100, (c) 500x500, (d) 1000x1000

For increasing resolution our gradient in the image continues to grow; therefore, we need to smooth out the *sharp* edges of the image. To do the smoothing with a given σ I

numerically solved the heat equation up to time $t = \sigma$, I utilized a Crank-Nicolson scheme in two dimensions analogous to LeVeque [10] assuming $h_x = h_y = h$:

$$\frac{U_{i,j}^{n+1} - U_{i,j}^{n}}{k} = \frac{1}{2}\left(\nabla^2 U_{i,j}^{n} + \nabla^2 U_{i,j}^{n+1}\right)$$

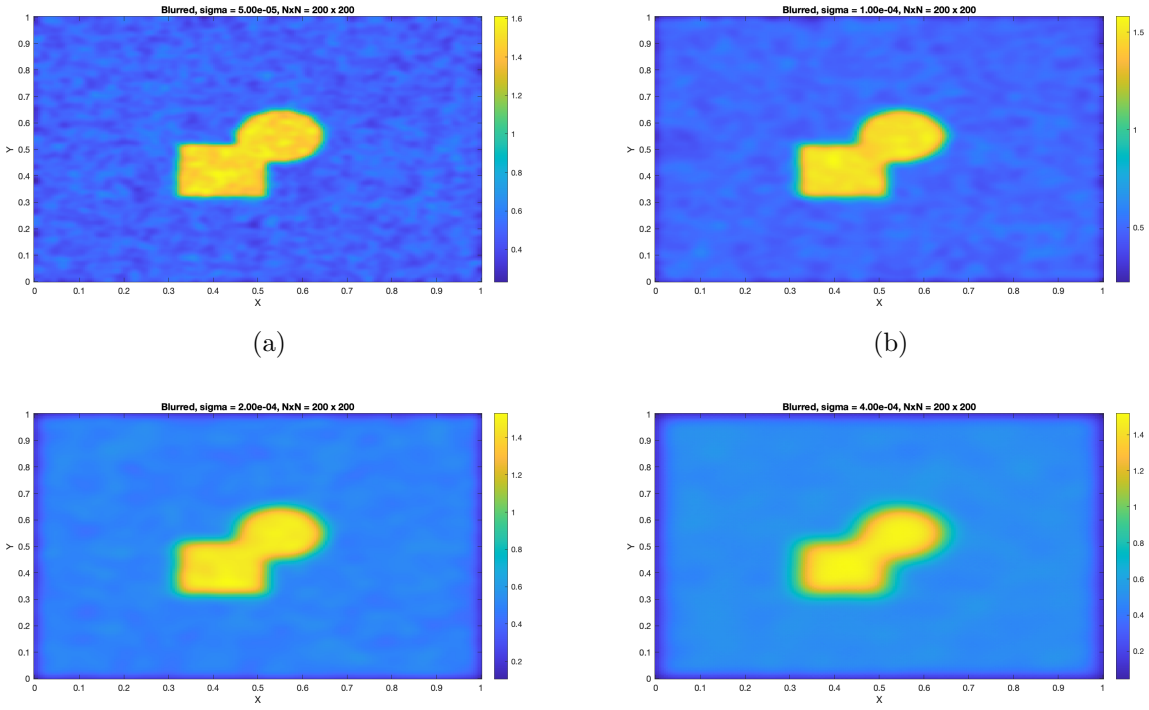
where

$$\nabla^2 U_{i,j}^{n} = \frac{U_{i-1,j}^{n} - 2U_{i,j}^{n} + U_{i+1,j}^{n}}{h^2} + \frac{U_{i,j-1}^{n} - 2U_{i,j}^{n} + U_{i,j+1}^{n}}{h^2}$$
$$\nabla^2 U_{i,j}^{n+1} = \frac{U_{i-1,j}^{n+1} - 2U_{i,j}^{n+1} + U_{i+1,j}^{n+1}}{h^2} + \frac{U_{i,j-1}^{n+1} - 2U_{i,j}^{n+1} + U_{i,j+1}^{n+1}}{h^2}$$

and I implemented Neumann boundary conditions $\partial U/\partial \eta = 0$. Below is the implementation code:

```
...
U = reshape(u,[],1); % initial conditions
% Matrix generation
K1D = spdiags(ones(nx,1)*[1 -2 1],-1:1,nx,nx)/h^2; % 1d Poisson matrix
I1D = speye(size(K1D)); % 1d identity matrix
K2D = kron(K1D,I1D)+kron(I1D,K1D); % 2d Poisson matrix
I2D = speye(size(K2D)); % 2d identity matrix
A = I2D-k/2*K2D; % LHS matrix for Crank-Nicolson
B = I2D+k/2*K2D; % RHS matrix for Crank-Nicolsons
ndpts = floor(3);
s = mit18086_stencil_stability((0:1:ndpts),1);
% Time loop
for j = 0:nt
    if j>0 % update step
        U = A\(B*U);
    end
    % for neumann b.c.
    U = weightedsum(U,s,nx);
end
U = reshape(U,nx,nx);
end
```

Below are images of the smoothed out noisey image for different variance σ:



(a) (b)

(c) (d)

Figure 3: Variance (a) 5×10^{-5}, (b) 1×10^{-4}, (c) 2×10^{-4}, (d) 4×10^{-4}

Another important point to address is how the variance σ and the width of the smoothing are related. To do this we can analytically solve the heat equation with Dirac initial data:

$$\begin{cases} u_t = -\nabla^2 u & \vec{x} \in \Omega, t > 0 \\ u(x,0) = \delta_0(x) \end{cases}$$

the solution to this is the fundamental solution [5]:

$$\Phi(\vec{x}, t) = \frac{1}{4\pi t} e^{-|\vec{x}|^2/(4t)}$$

the amount of smoothing is determined by $t = \sigma > 0$; therefore,

$$\Phi(\vec{x}, \sigma) = \frac{1}{4\pi\sigma} e^{-|\vec{x}|^2/(4\sigma)}$$

The maximum occurs when $\vec{x} = 0$; therefore, $\Phi_{max} = 1/(4\pi\sigma)$. Now the width of the smoothing can be given by when Φ is 5% of the maximum; therefore, we need to solve

$$0.05 \cdot \frac{1}{4\pi\sigma} = \frac{1}{4\pi\sigma} e^{-d^2/(4\sigma)}$$

which has solution given by

$$d = 2\sqrt{\sigma \ln(20)}.$$

As an example consider:

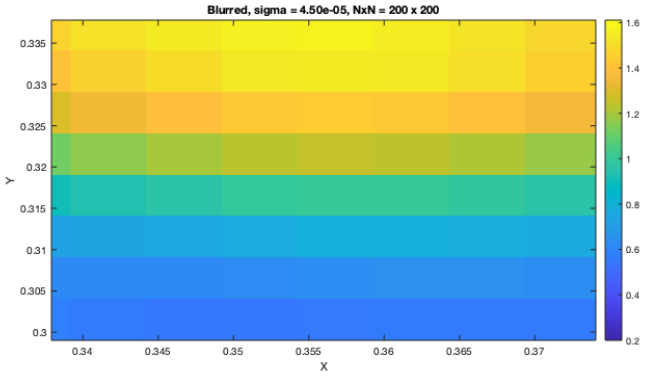


Figure 4: $\sigma = 4.5 \times 10^{-5}$

Here the variance $\sigma = 4.5 \times 10^{-5}$ can be plugged in to our relationship to get $d = 0.0232$ which is about the width of the smoothing region. Also we have that the grid resolution is $h = 1/200 < d$, therefore we capture the smoothing regions of our image. However if we have resolution more than the smoothing width we lose the smoothing region:

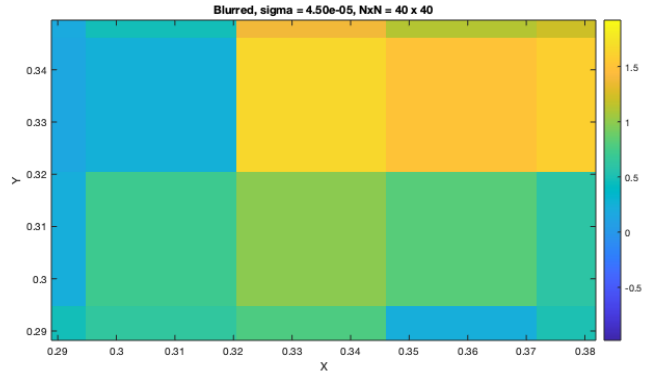


Figure 5: $\sigma = 4.5 \times 10^{-5}$ and $h = 1/40$

this will cause the level set front to encounter very high jumps, that is large gradients. Now we have a relationship between σ and d the smoothing width, this will allows us to carefully choose p and γ. We want p and γ to be such that $|\nabla g(\nabla u_0)|$ is order magnitude of $1/d$.

3. Numerical Method

To establish the numerical scheme, I will write out the terms for the *generalized curvature* given in our PDE equation

$$\underbrace{A\nabla \cdot \frac{\nabla \phi}{|\nabla \phi|}}_{\text{Part 1}} + \underbrace{\nabla A \cdot \frac{\nabla \phi}{|\nabla \phi|}}_{\text{Part 2}}$$

where $A = g(\nabla u_0)$, I will do this in two parts. For Part 1 we have, Part 1

$$
\begin{aligned}
&= A(\partial_x, \partial_y) \cdot \frac{(\phi_x, \phi_y)}{(\phi_x^2 + \phi_y^2)^{1/2}} \\
&= A\left(\partial_x\left(\frac{\phi_x}{(\phi_x^2+\phi_y^2)^{1/2}}\right) + \partial_y\left(\frac{\phi_y}{(\phi_x^2+\phi_y^2)^{1/2}}\right)\right) \\
&= A\left(\frac{|\nabla\phi|\phi_{xx} - \phi_x\frac{1}{|\nabla\phi|}(\phi_x\phi_{xx}+\phi_y\phi_{yx})}{|\nabla\phi|^2} + \frac{|\nabla\phi|\phi_{yy} - \phi_y\frac{1}{|\nabla\phi|}(\phi_x\phi_{xy}+\phi_y\phi_{yy})}{|\nabla\phi|^2}\right) \\
&= \frac{A}{|\nabla\phi|^3}\left(|\nabla\phi|^2\phi_{xx} - \phi_x^2\phi_{xx} - 2\phi_x\phi_y\phi_{xy} - \phi_y^2\phi_{yy} + |\nabla\phi|^2\phi_{yy}\right)
\end{aligned}
$$

which is coded as

```
F1=(A.*((Pxx.*Py.^2-Px.*Py.*Pxy)+...
    (Pyy.*Px.^2-Px.*Py.*Pxy))./((Px.^2+Py.^2).^(1.5));
```

For the Part 2, I do an entry wise finite difference to compute A_x and A_y

```
Ax = (A(3:end,:)-A(1:end-2,:))/(2*h);
Ax = Ax([1 1:end end],:);
Ay = (A(:,3:end)-A(:,1:end-2))/(2*h);
Ay = Ay(:,[1 1:end end]);
```

then I add both terms

```
F2=(Ax.*Px+Ay.*Py)./((Px.^2+Py.^2).^(0.5));
F=(F1+F2);
F = min(max(F,-spd/h),spd/h);
```

note that I have a parameter "spd" to control the speed of the front, as it is possible for the level set to develop sharp cut-ins, this means that the front is moving too fast in some locations. The remaining multiplication of $|\nabla\phi|$ is treated as

$$|\nabla\phi| = (\phi_x^2 + \phi_y^2)^{1/2}$$

and is multiplied by the generalized curvature which I call "F" and the code is below:

```
DxP = diff(P)/h;    DxmP = DxP([1 1:end],:); DxpP = DxP([1:end end],:);
DyP = diff(P')'/h; DymP = DyP(:,[1 1:end]); DypP = DyP(:,[1:end end]);
Np = sqrt(max(DxmP,0).^2+min(DxpP,0).^2+max(DymP,0).^2+min(DypP,0).^2);
Nm = sqrt(min(DxmP,0).^2+max(DxpP,0).^2+min(DymP,0).^2+max(DypP,0).^2);
dP = max(F,0).*(Np-c)+min(F,0).*(Nm-c);
```

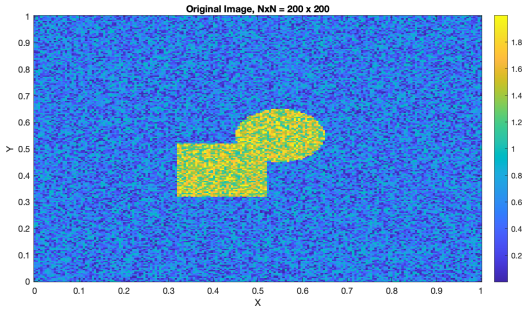
Then I use a FE update rule taking into account the signage of the curvature term:

```
for it = 1:nt
    F = -curvature(A,P,h,spd)*5e-3;      % movement under curvature
    P = P-dt*FabsgradP(P,h,F);       % level set update
end
```

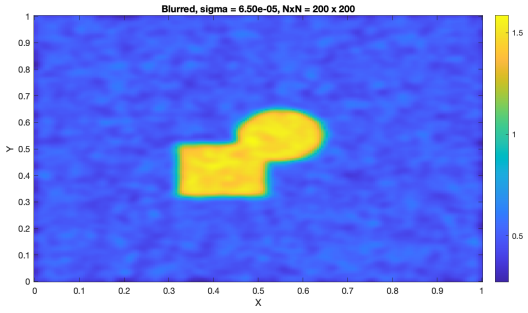
For the edge detector (2) I have two input parameters which will contribute to the movement of the level set front, the two parameters are given by γ and p. In the next section I demonstrate the results for different values of γ and p and also for different variance σ.

4. Results

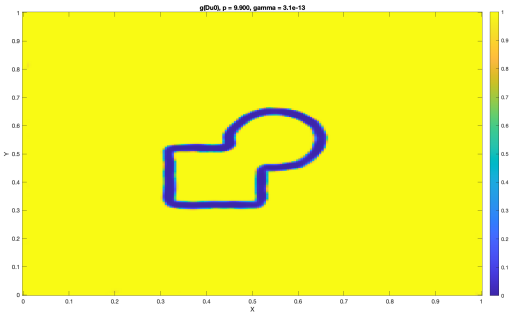
For this section I demonstrate the results for different parameters of p, γ, and σ. Below is an example that demonstrates the original image and the resolved image If I divide p in



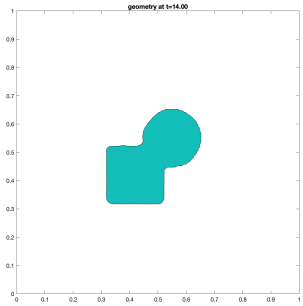
(a)



(b)



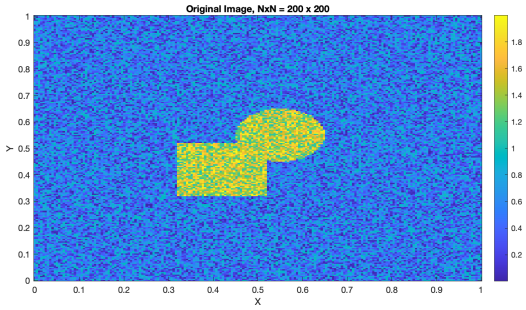
(c)



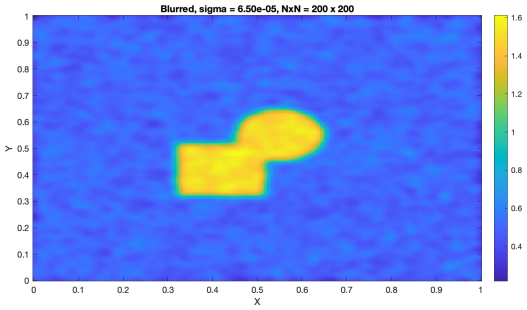
(d)

Figure 6: $\sigma = 6.5 \times 10^{-5}$, $p = 9.9$, $\gamma = 5.5 \times 10^{-13.25}$

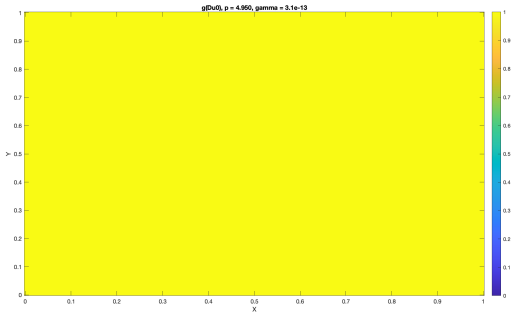
half I get



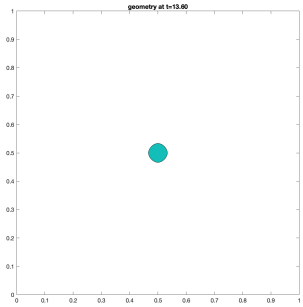
(a)



(b)



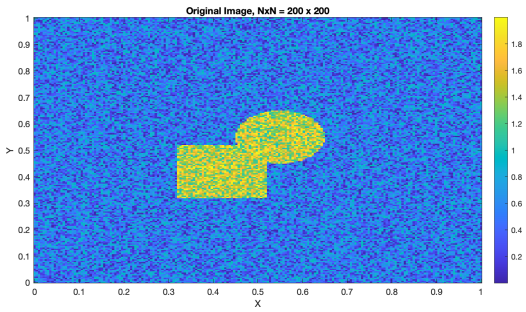
(c)



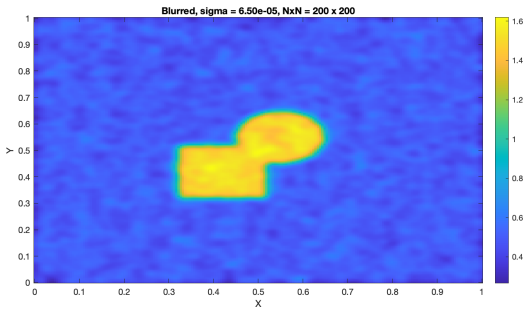
(d)

Figure 7: $\sigma = 6.5 \times 10^{-5}$, $p = 4.95$, $\gamma = 5.5 \times 10^{-13.25}$

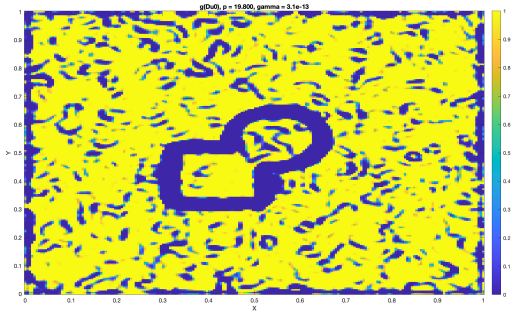
The middle plot that is yellow indicates that the front will continue to move at speed 1; therefore, this parameter was too low for the front to "detect" the image contour. In the next example I double p



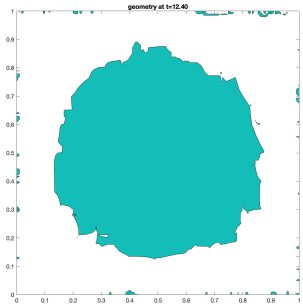
(a)



(b)



(c)

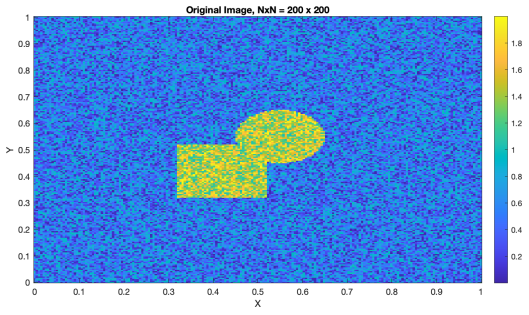


(d)

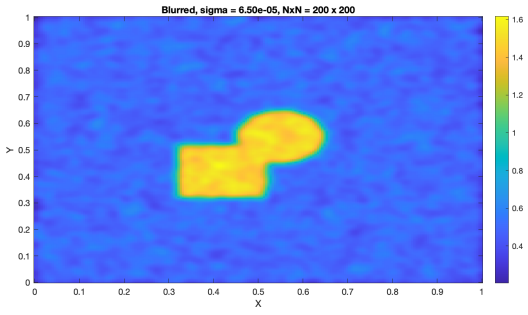
Figure 8: $\sigma = 6.5 \times 10^{-5}$, $p = 19.8$, $\gamma = 5.5 \times 10^{-13.25}$

For the example above, the gradients are enhanced, which causes the front to move erratically; that is some parts of the function move inward while some are moving outward. We also see artifacts appearing along the edges of the image.

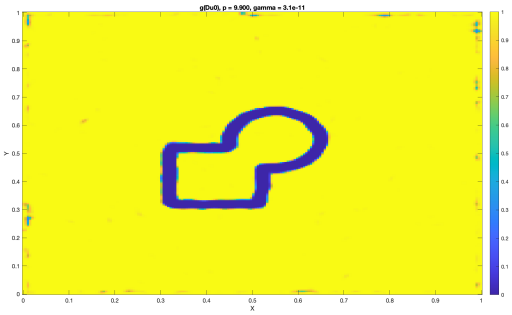
For this example I fixed $p = 9.9$ and multiplied γ by 100: This caused a slight increase



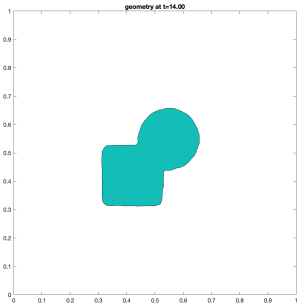
(a)



(b)



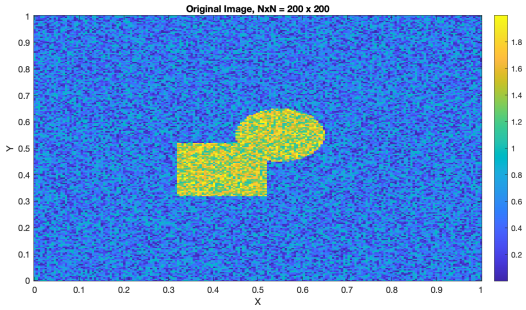
(c)



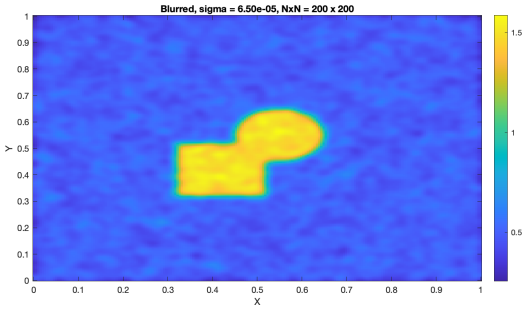
(d)

Figure 9: $\sigma = 6.5 \times 10^{-5}$, $p = 9.9$, $\gamma = 5.5 \times 10^{-11.25}$

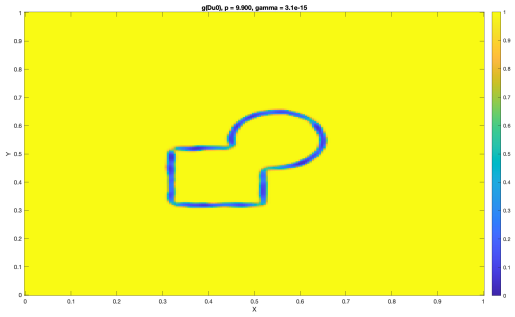
in the speed at noisey locations near the edge of the image, but the front still converged to the image.



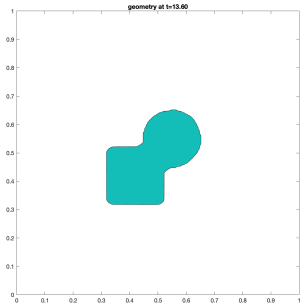
(a)



(b)



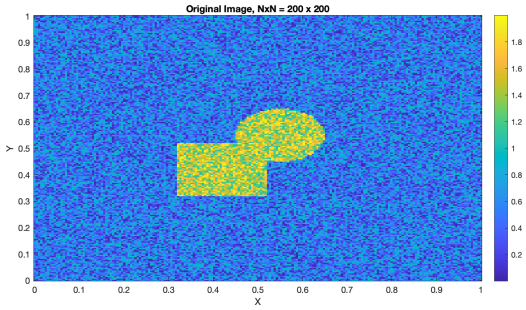
(c)



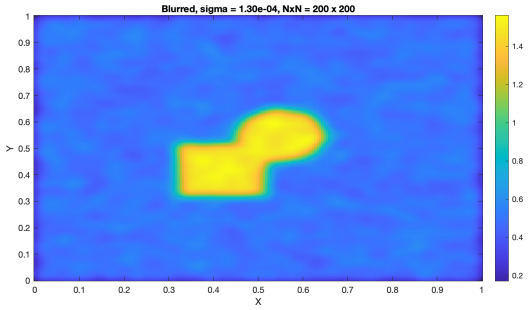
(d)

Figure 10: $\sigma = 6.5 \times 10^{-5}$, $p = 9.9$, $\gamma = 5.5 \times 10^{-15.25}$

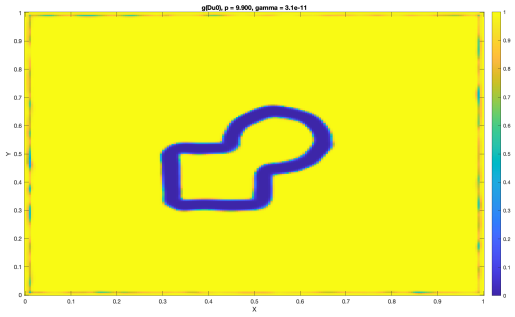
In this example I divided γ by a factor of 100. The speed increased at the interface of the image; however the image still converged and did not 'pass' through. I also ran the code with more smoothing of the image, I doubled the variance:



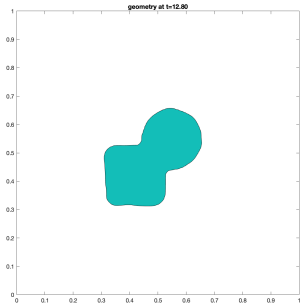
(a)



(b)



(c)



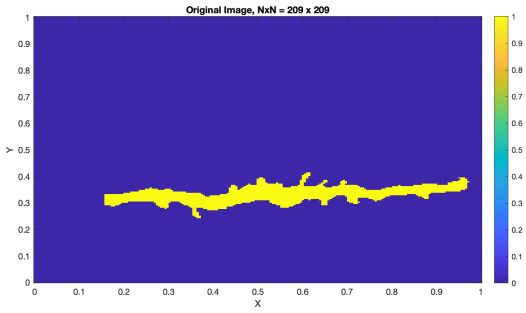
(d)

Figure 11: $\sigma = 13 \times 10^{-5}$, $p = 9.9$, $\gamma = 5.5 \times 10^{-15.25}$

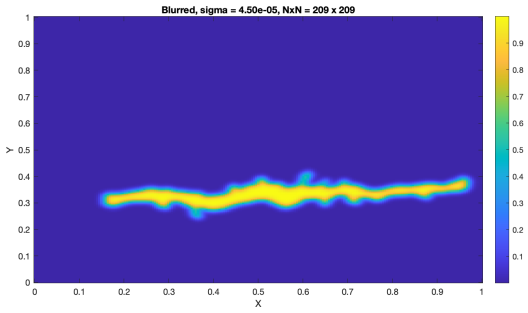
This caused the contour to stop "sooner" and therefore the image is wider in some places compared to the original image. I then ran a test on a spine-dendrite image:



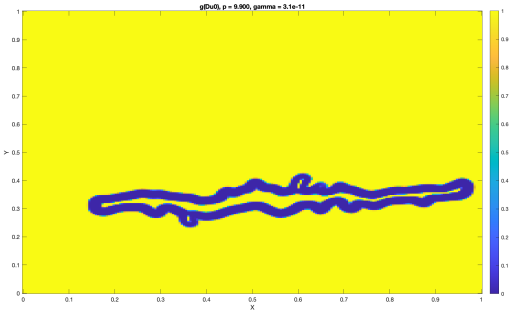
I modified the level set function so that the contour would begin outside the image.



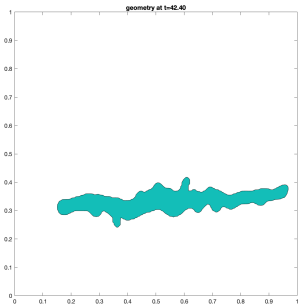
(a)



(b)



(c)



(d)

Figure 12: $\sigma = 13 \times 10^{-5}$, $p = 9.9$, $\gamma = 5.5 \times 10^{-11.25}$

I was able to use the same parameters as before and still capture the spines that are protruding from the dendrite.

5. Conclusion

In this section I will discuss the challenges faced in the project and the parts of the project that need refinement and/or completion. One of the challenges faced in this project is properly defining the edge detector for the moving image front, in particular making it robust against noise. For instance, what parameters of p and γ and the amount of smoothing are necessary to ensure that the front continues to "move" towards the image and not get stuck on a noisy section. If enough Gaussian smoothing is applied then the noisy part smooths away; however, this is at the cost of destroying feature and landmarks on the image we wish to detect. The choice of parameters γ and p depend on the smoothing amount and also on the image resolution, if the image is too poor and too much smoothing then the moving front has no chance of constructing the image features. These scenarios involve trial error to determine the "best" parameters so that the image is reasonably reconstructed. Some items that need to be address are

- Implement the Mumford-Shah functional [1]

$$E[J,B] = C\int_D (I(\vec{x}) - J(\vec{x}))^2\,\mathrm{d}\vec{x} + A\int_{D/B} \vec{\nabla}J(\vec{x})\cdot\vec{\nabla}J(\vec{x})\,\mathrm{d}\vec{x} + B\int_B ds$$

  optimizing this functional leads to a criteria for segmenting an image into sub-image regions and Ambrosio-Tortorelli give an algorithm for achieving the minimum and also show the minimum is well-defined.

- Implement the gradient vector flow methodology

- Test the two implementations above on 2d neuron geometries e.g. and 2d graph essentially

- Then implement on 3d geometries of neurons

References

[1] L. Ambrosio and V. M. Tortorelli, *Approximation of functional depending on jumps by elliptic functional via t-convergence*, Communications on Pure and Applied Mathematics, 43 (1990), pp. 999–1036, https://doi.org/10.1002/cpa.3160430805.

[2] J. Canny, *A computational approach to edge detection*, IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8 (1986), pp. 679–698, https://doi.org/10.1109/TPAMI.1986.4767851.

[3] V. Caselles, R. Kimmel, and G. Sapiro, *Geodesic active contours*, 1997.

[4] Chenyang Xu and J. L. Prince, *Gradient vector flow: a new external force for snakes*, in Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition, June 1997, pp. 66–71, https://doi.org/10.1109/CVPR.1997.609299.

[5] L. Evans, *Partial Differential Equations*, Graduate studies in mathematics, American Mathematical Society, 2010, https://books.google.com/books?id=Xnu0o_EJrCQC.

[6] R. A. Haddad and A. N. Akansu, *A class of fast gaussian binomial filters for speech and image processing*, IEEE Transactions on Signal Processing, 39 (1991), pp. 723–727, https://doi.org/10.1109/78.80892.

[7] M. Kass, A. Witkin, and D. Terzopoulos, *Snakes: Active contour models*, International Journal of Computer Vision, 1 (1988), pp. 321–331.

[8] S. Kichenassamy, A. Kumar, P. Olver, A. Tannenbaum, and A. Yezzi, *Conformal curvature flows: From phase transitions to active vision*, Archive for Rational Mechanics and Analysis, 134 (1996), pp. 275–301, https://doi.org/10.1007/BF00379537.

[9] R. Klette, *Concise Computer Vision: An Introduction into Theory and Algorithms*, Undergraduate Topics in Computer Science, Springer London, 2014, https://books.google.com/books?id=ZCu8BAAAQBAJ.

[10] R. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, Other Titles in Applied Mathematics, Society for Industrial and Applied Mathematics, 2007, https://books.google.com/books?id=qsvmsXe8Ug4C.

[11] T. Morris, *Computer Vision and Image Processing*, Cornerstones of Computing, Macmillan Education UK, 2003, https://books.google.com/books?id=MQGAQgAACAAJ.

[12] M. Nixon and A. Aguado, *Feature Extraction and Image Processing*, Electronics & Electrical, Newnes, 2002, https://books.google.com/books?id=3JuesLhNXBAC.

[13] S. Osher and R. Fedkiw, *Level Set Methods and Dynamic Implicit Surfaces*, Applied Mathematical Sciences, Springer New York, 2006, https://books.google.com/books?id=i4bfBwAAQBAJ.

[14] L. Shapiro and G. Stockman, *Computer Vision*, Prentice Hall, 2001, https://books.google.com/books?id=FftDAQAAIAAJ.

[15] G. Stockman and L. G. Shapiro, *Computer Vision*, Prentice Hall PTR, USA, 1st ed., 2001.

[16] H.-K. Zhao, T. Chan, B. Merriman, and S. Osher, *A variational level set approach to multiphase motion*, Journal of Computational Physics, 127 (1996), https://doi.org/10.1006/jcph.1996.0167.

Visualizing Neurons: A Comparative Analysis of PDE-Based Image Reconstruction

James Rosado

January 23, 2020

Project Proposal

Image segmentation is the process of partitioning raw digital data (an image) into image objects, e.g. pixels, line segments, closed contours/surfaces. The purpose is to generate meaningful data e.g. a 2D closed contour or 3D triangulation that then can be utilized as a computational domain to analyze/solve PDE model equations on [4]. The process itself will require the use of non-linear equations PDEs. In particular one methodology utilizes active contours (or snakes) [3], that is an evolving curve which is constrained by a given raw image $u_0(x)$, where $x \in \Omega \subset \mathbb{R}^2$ in 2D as an example. The snake models that are used [3] are based on

$$F_1(C) = \alpha \int_0^1 |C'(s)|^2 \, ds + \beta \int_0^1 |C''(s)| \, ds - \lambda \int_0^1 |\nabla u_0(C(s))|^2 \, ds$$

where the contour C that minimizes the above functional is the contour that realizes the image and this is analogous to a non-linear PDE. A more compact version [3] is given by

$$F_2(C) = \int_0^1 g(\nabla(u_0(s))) \, ds$$

which is analogous to

$$\phi_t = |\nabla \phi| g(\nabla u_0) \kappa + \nabla g(\nabla u_0) \cdot \nabla \phi$$

where ϕ are the isocontours, level sets, of our image u_0, this formulation is given by [5]. Another methodology involves gradient vector flows [2]

$$E_{\text{GVF}} = \iint \mu(u_x^2 + u_y^2 + v_x^2 + v_y^2) + |\nabla f|^2 |\mathbf{v} - \nabla f|^2 \, dx \, dy$$

where μ is a controllable smoothing term, which is solve by the Euler equations

$$\mu \nabla^2 u - \left(u - \frac{\partial}{\partial x} F_{\text{ext}} \right) \left(\frac{\partial}{\partial x} F_{\text{ext}}(x,y)^2 + \frac{\partial}{\partial y} F_{\text{ext}}(x,y)^2 \right) = 0$$

$$\mu \nabla^2 v - \left(v - \frac{\partial}{\partial y} F_{\text{ext}} \right) \left(\frac{\partial}{\partial x} F_{\text{ext}}(x,y)^2 + \frac{\partial}{\partial y} F_{\text{ext}}(x,y)^2 \right) = 0$$

A third methodology is to use the Mumford-Shah functional [1]

$$E[J,B] = C \int_D (I(\vec{x}) - J(\vec{x}))^2 \, \mathrm{d}\vec{x} + A \int_{D/B} \vec{\nabla} J(\vec{x}) \cdot \vec{\nabla} J(\vec{x}) \, \mathrm{d}\vec{x} + B \int_B ds$$

optimizing this functional leads to a criteria for segmenting an image into sub-image regions and Ambrosio-Tortorelli give an algorithm for acheiving the minimum and also show the minimum is well-defined. For this project we will implement the aforementioned methodologies to realize computational domains from 2D and then 3D raw neuronal cell data. From these implementations we will perform a comparative analysis of the reproduced image data and determine if there is a particular methodology that is more efficient given the unstructured nature of neuron cells.

References

[1] Luigi Ambrosio and Vincenzo Maria Tortorelli. Approximation of functional depending on jumps by elliptic functional via t-convergence. *Communications on Pure and Applied Mathematics*, 43(8):999–1036, 1990.

[2] Chenyang Xu and J. L. Prince. Gradient vector flow: a new external force for snakes. In *Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 66–71, June 1997.

[3] S. Osher and R. Fedkiw. *Level Set Methods and Dynamic Implicit Surfaces*. Applied Mathematical Sciences. Springer New York, 2006.

[4] George Stockman and Linda G. Shapiro. *Computer Vision*. Prentice Hall PTR, USA, 1st edition, 2001.

[5] Hong-Kai Zhao, T. Chan, B. Merriman, and S. Osher. A variational level set approach to multiphase motion. *Journal of Computational Physics*, 127(1), 8 1996.

Visualizing Neurons: A Comparative Analysis of PDE-Based Image Reconstruction

James Rosado

March 9, 2020

Midterm Project Report

I have currently implemented a numerical method that utilizes active contours (or snakes) [5], which is constrained by a given raw image $u_0(x)$, where $x \in \Omega \subset \mathbb{R}^2$ in 2D as an example. The snake models that are used [5] are based on

$$F_1(C) = \alpha \int_0^1 |C'(s)|^2 \, ds + \beta \int_0^1 |C''(s)| \, ds - \lambda \int_0^1 |\nabla u_0(C(s))|^2 \, ds$$

where the contour C that minimizes the above functional is the contour that realizes the image and this is analogous to a non-linear PDE. A more compact version [5] is given by

$$F_2(C) = \int_0^1 g(\nabla(u_0(s))) \, ds$$

which is analogous to

$$\phi_t = |\nabla \phi| g(\nabla u_0) \kappa + \nabla g(\nabla u_0) \cdot \nabla \phi \tag{1}$$

where ϕ are the isocontours, level sets, of our image u_0, this formulation is given by [6]. This is also a variation of the formulation for Geometric active contour [2], or geodesic active contour [4]. This method employs Euclidean curvature shortening evolution and the isocontours split and merge depending on the detection of objects in the image, for the detection I use Gaussian blurring. Below is the corresponding formulation:

$$\frac{\partial C}{\partial t} = g(I)(c + \kappa)\vec{N} - \langle \nabla g, \vec{N} \rangle \vec{N},$$

where κ is the curvature, g is the edge detector also called the halting function, c is a Lagrange multiplier, and $\vec{N}$ is the inward unit normal. For my initial attempt at the level set method I implemented the update rule initially with only the advection term and then I did an implementation with the advection and curvature term. Below is the update rule I utilize:

$$\frac{\phi_{i,j}^{n+1} - \phi_{i,j}^n}{k}$$
$$= \sqrt{\left(\frac{\phi_{i,j}^n - \phi_{i,j-1}^n}{h} \right)^2 + \left(\frac{\phi_{i,j}^n - \phi_{i-1,j}^n}{h} \right)^2} [g(\nabla u_0)]_{i,j} \kappa_{i,j}(u_0) + ([g_x(\nabla u_0)]_{i,j}, [g_y(\nabla u_0)]_{i,j}) \cdot \left(\frac{\phi_{i,j}^n - \phi_{i-1,j}^n}{h}, \frac{\phi_{i,j}^n - \phi_{i,j-1}^n}{h} \right)$$

The formulation given by [6] is based on level sets and follows from the theorem:

Theorem 1 *The solutions to the minimization problem: minimize $E_1 + E_2$ in a fixed domain D subject to integral constraint*

$$\iint \frac{(\sum H(\phi_i(x, y, t)) - 1)^2}{2} \, dx \, dy = \epsilon$$

satisfying for $i = 1, \ldots, n$

$$\delta(\phi_i) \left(\gamma_i \nabla \cdot \left(\frac{\nabla \phi_i}{|\nabla \phi_i|} \right) - e_i - \lambda \left(\sum_{i=1}^n H(\phi_i) - 1 \right) \right) = 0$$

with boundary conditions

$$\frac{\delta(\phi_i)}{|\nabla \phi_i|} \frac{\partial \phi_i}{\partial n} = 0$$

on ∂D, where λ is a Lagrange multiplier.

For reference $E_1 + E_2$ is given by

$$E_1 = \sum_{i=1}^{n} \gamma_i \iint \delta(\phi_i(x,y,t))|\nabla\phi_i(x,y,t)| \; dx \; dy$$
$$E_2 = \sum_{i=1}^{n} e_i \iint H(\phi_i(x,y,t)) \; dx \; dy.$$

Below is the initial image and the corresponding Gaussian blurred image for edge detection:

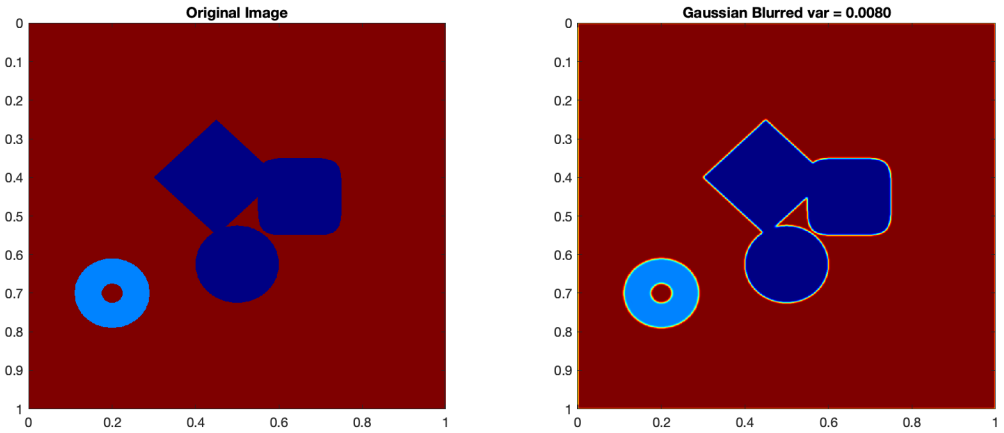


Figure 1: The figure on the left is the original image, on the right we have blurring with variance $\sigma = 0.008$.

Below are the time evolutions: I did two sets of plots to observe the difference between including and not

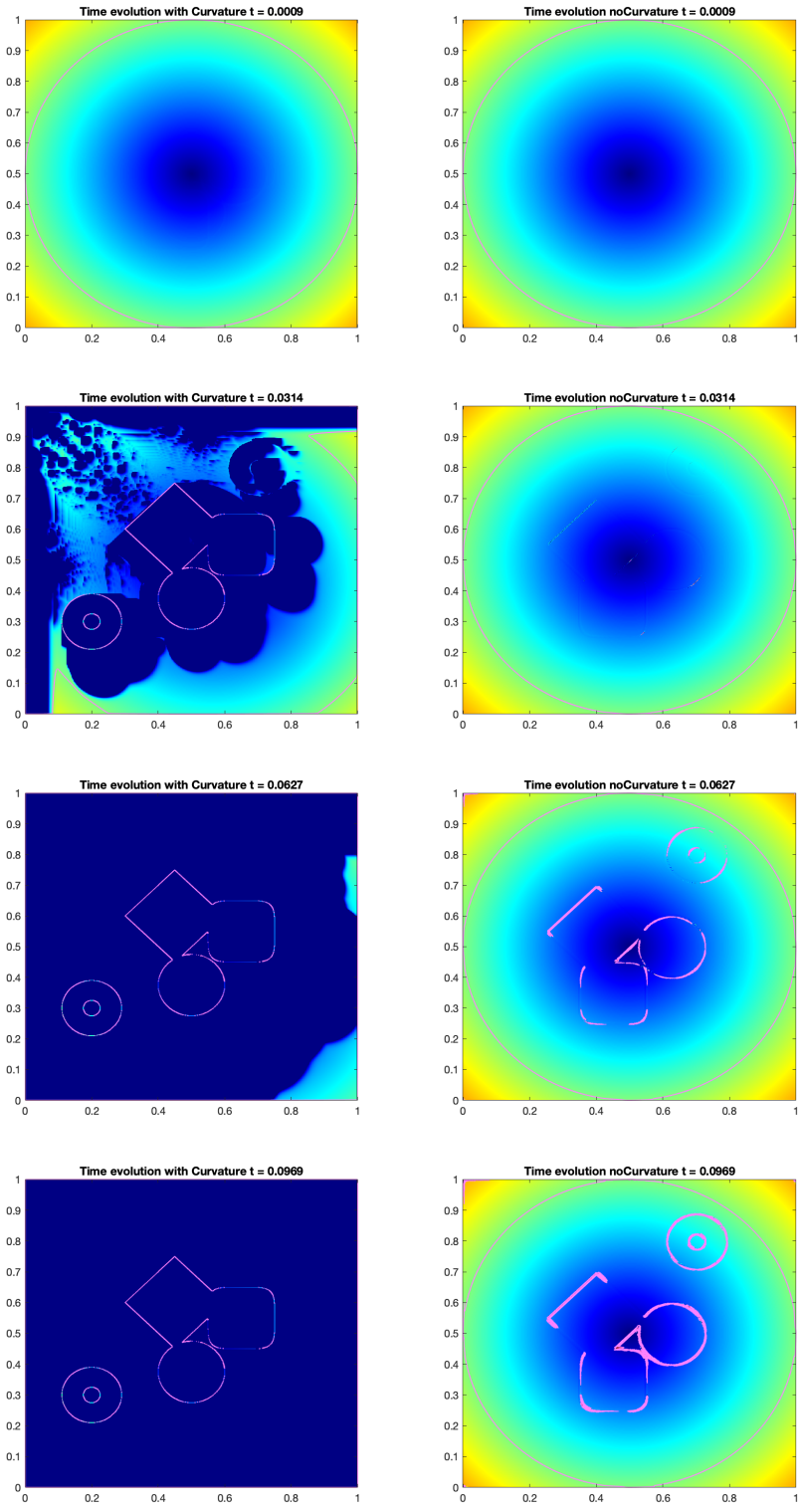


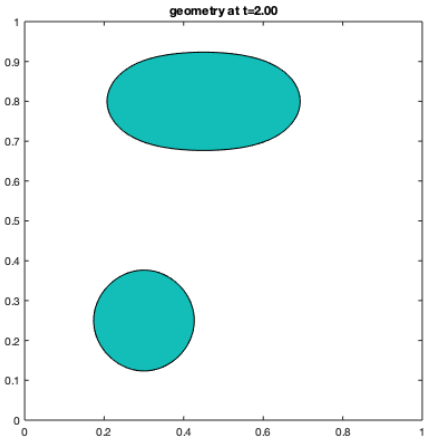
Table 1: The left plots have curvature, the right plots do not have curvature

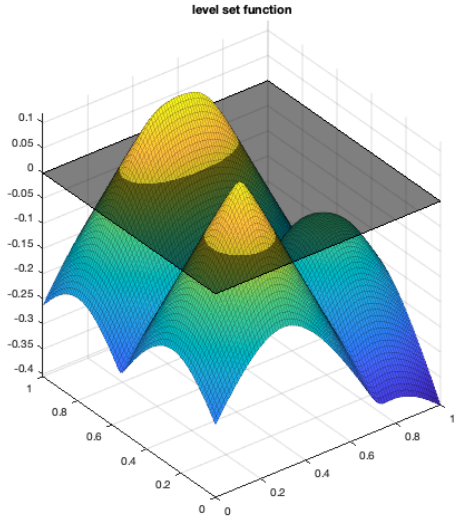
including the curvature term. Below is a snippet of the code of the update rule:

```
for n=1:nT
    for i=2:npts-1
        for j=2:npts-1
            phi(i,j)=cfl*sqrt((phi(i,j)-phi(i,j-1))^2+(phi(i,j)-phi(i-1,j))^2)*g_DU(i,j)*...
            kurv(i,j)+(1+cfl*(Dxg_DU(i,j)+Dyg_DU(i,j)))*phi(i,j)-cfl*(Dxg_DU(i,j)*...
            phi(i-1,j)+Dyg_DU(i,j)*phi(i,j-1));

            phi2(i,j)=(1+cfl*(Dxg_DU(i,j)+Dyg_DU(i,j)))*phi2(i,j)-cfl*(Dxg_DU(i,j)*...
            phi2(i-1,j)+Dyg_DU(i,j)*phi2(i,j-1));
        end
    end
```

This implementation did not work in that you do not see the isocontour "move" towards the shape of the raster image; instead, the contours develop separately along with other numerical artifacts, such as jumps and holes. For my second implementation I will modify the levelset font code, initially it computes the movement of fronts under a given velocity field:





First, our original formulation (1) can be re-written as

$$\phi_t = |\nabla\phi| \underbrace{\nabla \cdot \left(g(\nabla u_0) \left(\frac{\nabla\phi}{|\nabla\phi|} \right) \right)}_{F}$$

where F is the generalized curvature. The code for levelset front solves the problem $\phi_t + F|\nabla\phi| = 0$. Therefore, I will modify the definition of F in the code to accept the generalized curvature for F.

Another methodology I have to investigate involves gradient vector flows [3]

$$E_{\text{GVF}} = \iint \mu(u_x^2 + u_y^2 + v_x^2 + v_y^2) + |\nabla f|^2 |\mathbf{v} - \nabla f|^2 \, dx \, dy$$

where μ is a controllable smoothing term, which is solved by the Euler equations

$$\mu \nabla^2 u - \left(u - \frac{\partial}{\partial x} F_{\text{ext}} \right) \left(\frac{\partial}{\partial x} F_{\text{ext}}(x,y)^2 + \frac{\partial}{\partial y} F_{\text{ext}}(x,y)^2 \right) = 0$$

$$\mu \nabla^2 v - \left(v - \frac{\partial}{\partial y} F_{\text{ext}} \right) \left(\frac{\partial}{\partial x} F_{\text{ext}}(x,y)^2 + \frac{\partial}{\partial y} F_{\text{ext}}(x,y)^2 \right) = 0$$

A third methodology is to use the Mumford-Shah functional [1]

$$E[J,B] = C\int_D (I(\vec{x}) - J(\vec{x}))^2 \,\mathrm{d}\vec{x} + A\int_{D/B} \vec{\nabla} J(\vec{x}) \cdot \vec{\nabla} J(\vec{x}) \,\mathrm{d}\vec{x} + B\int_B ds$$

optimizing this functional leads to a criteria for segmenting an image into sub-image regions and Ambrosio-Tortorelli give an algorithm for acheiving the minimum and also show the minimum is well-defined. To summarize:

- Continue investigating the level set method described by [6]
- Implement the gradient vector flow methodology
- Test the two implementations above on 2d neuron geometries e.g. and 2d graph essentially
- Then implement on 3d geometries of neurons
- Implement an algorithm for minimizing the Mumford-Shah functional.

References

[1] Luigi Ambrosio and Vincenzo Maria Tortorelli. Approximation of functional depending on jumps by elliptic functional via t-convergence. *Communications on Pure and Applied Mathematics*, 43(8):999–1036, 1990.

[2] Vicent Caselles, Ron Kimmel, and Guillermo Sapiro. Geodesic active contours, 1997.

[3] Chenyang Xu and J. L. Prince. Gradient vector flow: a new external force for snakes. In *Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 66–71, June 1997.

[4] Satyanad Kichenassamy, Arun Kumar, Peter Olver, Allen Tannenbaum, and Anthony Yezzi. Conformal curvature flows: From phase transitions to active vision. *Archive for Rational Mechanics and Analysis*, 134:275–301, 01 1996.

[5] S. Osher and R. Fedkiw. *Level Set Methods and Dynamic Implicit Surfaces*. Applied Mathematical Sciences. Springer New York, 2006.

[6] Hong-Kai Zhao, T. Chan, B. Merriman, and S. Osher. A variational level set approach to multiphase motion. *Journal of Computational Physics*, 127(1), 8 1996.